8051 Timers

- The 8051 has two timers/counters, they can be used as
  - Timers to generate a time delay
  - Event counters to count events happening outside the microcontroller
- Both Timer 0 and Timer 1 are 16 bits wide
  - Since 8051 has an 8-bit architecture, each 16-bits timer is accessed as two separate registers of low byte and high byte

Timer 0 & 1 Registers

- Accessed as low byte and high byte
  - The low byte register is called TL0/TL1
  - The high byte register is called TH0/TH1
  - Accessed like any other register

  MOV TL0, #4FH
  MOV R5, TH0

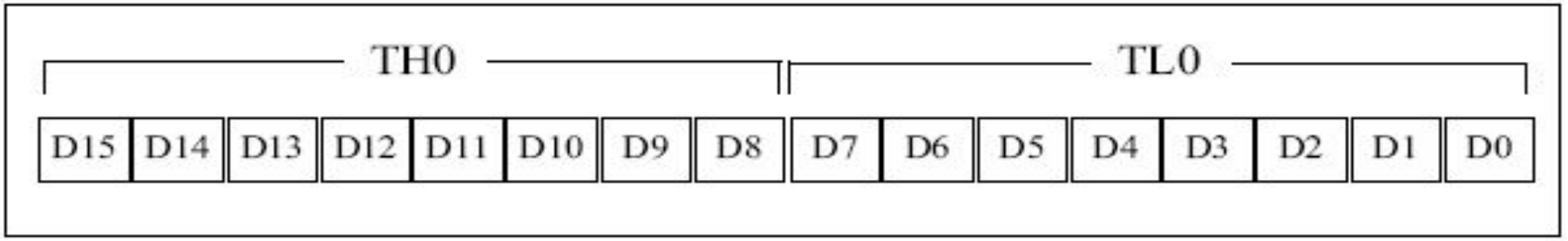


Figure 9-1. Timer 0 Registers

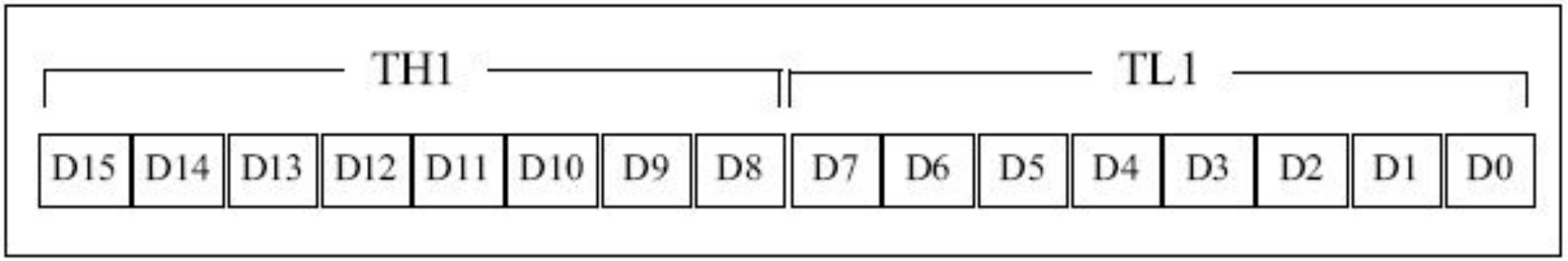


Figure 9-2. Timer 1 Registers

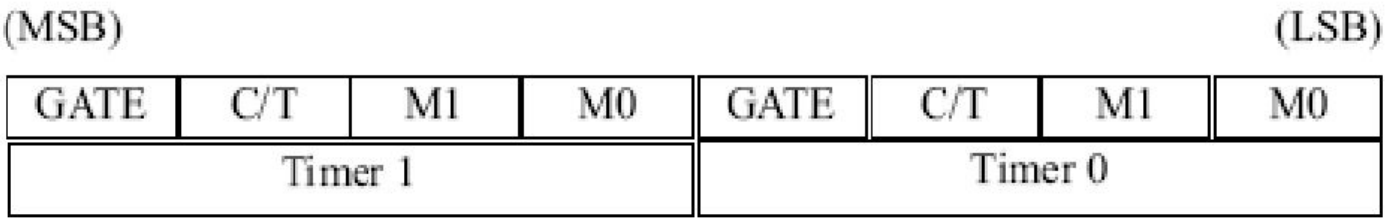
TMOD Register

- Both timers 0 and 1 use the same register, called TMOD (timer mode), to set the various timer operation modes
  - TMOD is a 8-bit register
    - The lower 4 bits are for Timer 0
    - The upper 4 bits are for Timer 1
      - In each case, the lower 2 bits are used to set the timer mode
      - The upper 2 bits to specify the operation

Gate

Every timer has a mean of starting and stopping.

- GATE=0
  - **Internal** control
  - The start and stop of the timer are controlled by way of **software.**
  - Set/clear the TR for start/stop timer. SETB TR0 & CLR TR0
- GATE=1
  - **External** control
  - The hardware way of starting and stopping the timer by **software** and **an external source**.
  - Timer/counter is enabled only while the INT pin is high and the TR control pin is set (TR).



GATE Gating control when set. The timer/counter is enabled only while the INTx pin is high and the TRx control pin is set. When cleared, the timer is enabled whenever the TRx control bit is set.

C/T Timer or counter selected cleared for timer operation (input from internal system clock). Set for counter operation (input from Tx input pin).

M1 Mode bit 1

M0 Mode bit 0

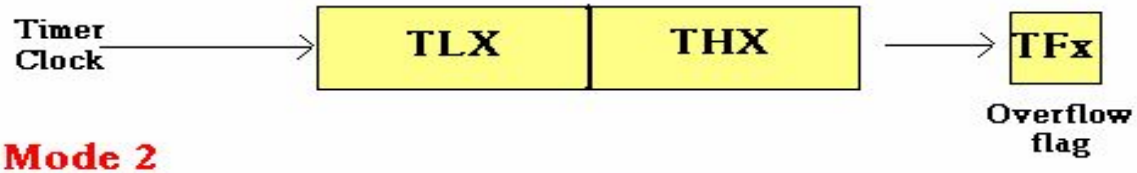
M1	M0	Mode	Operating Mode
0	0	0	13-bit timer mode 8-bit timer/counter THx with TLx as 5-bit prescaler
0	1	1	16-bit timer mode 16-bit timer/counters THx and TLx are cascaded; there is no prescaler
1	0	2	8-bit auto reload 8-bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows.
1	1	3	Split timer mode

Figure 9-3. TMOD Register

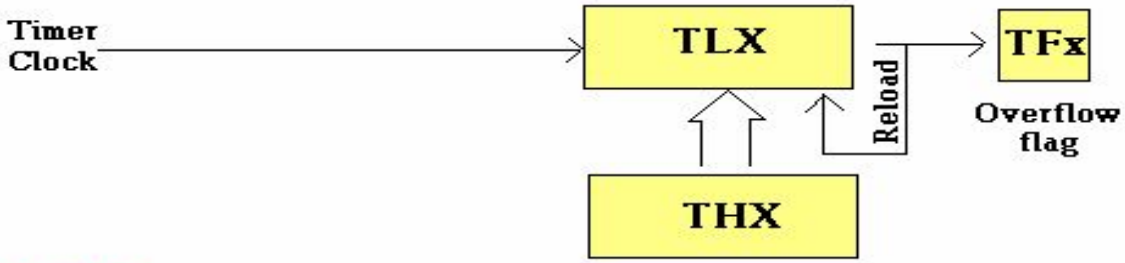
Mode 0



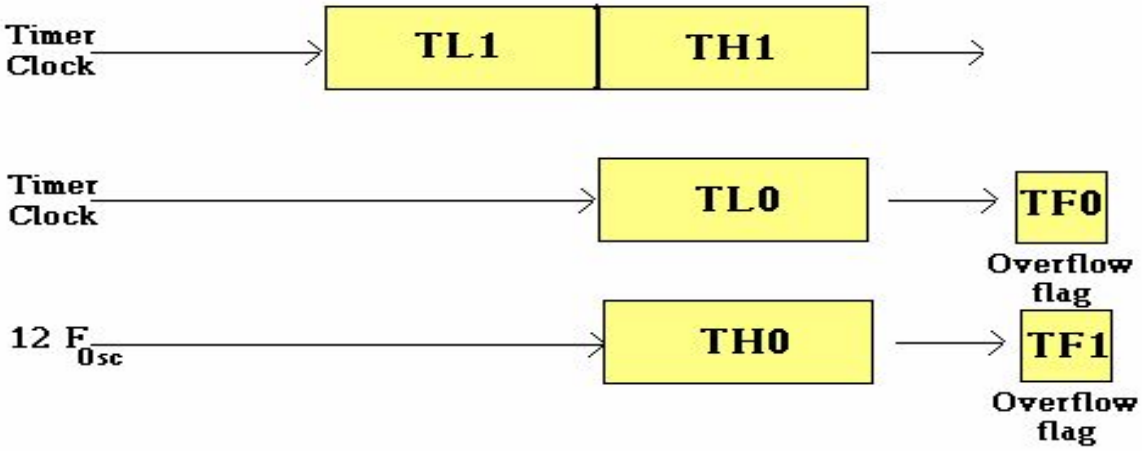
Mode 1



Mode 2

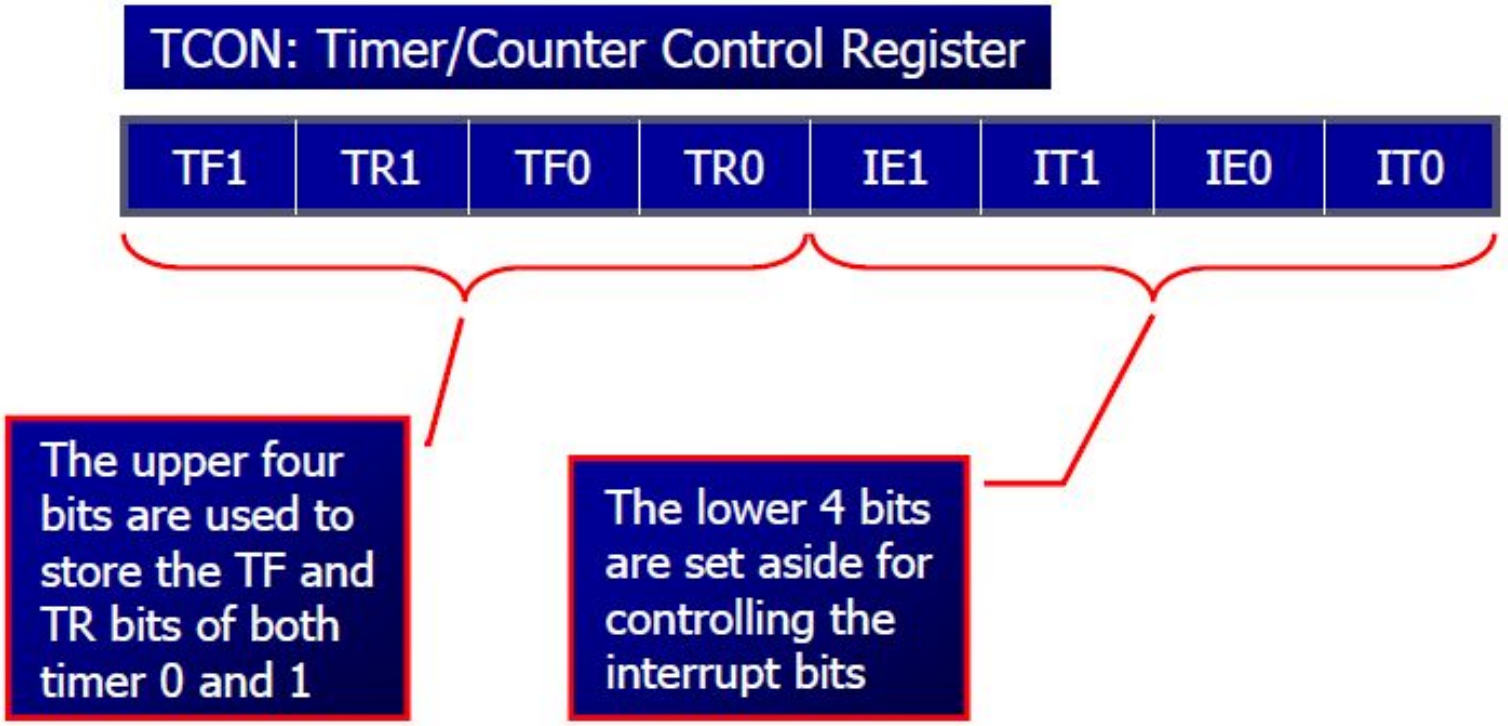


Mode 3



TCON Register

- TCON (timer control) register is an 8-bit register



TCON Register (cont.)

- TCON register is a bit-addressable register

Equivalent instruction for the Timer Control Register			
For timer 0			
SETB	TR0	=	SETB TCON.4
CLR	TR0	=	CLR TCON.4
SETB	TF0	=	SETB TCON.5
CLR	TF0	=	CLR TCON.5
For timer 1			
SETB	TR1	=	SETB TCON.6
CLR	TR1	=	CLR TCON.6
SETB	TF1	=	SETB TCON.7
CLR	TF1	=	CLR TCON.7

Example 9-1

Indicate which mode and which timer are selected for each of the following.

(a) `MOV TMOD,#01H` (b) `MOV TMOD,#20H` (c) `MOV TMOD,#12H`

Solution:

We convert the values from hex to binary. From Figure 9-3 we have:

(a) TMOD = 00000001, mode 1 of Timer 0 is selected.
(b) TMOD = 00100000, mode 2 of Timer 1 is selected.
(c) TMOD = 00010010, mode 2 of Timer 0, and mode 1 of Timer 1 are selected.

GATE

- Timers of 8051 do starting and stopping by either software or hardware control
  - In using software to start and stop the timer where GATE=0
    - The start and stop of the timer are controlled by way of software by the TR (timer start) bits TR0 and TR1
    - The SETB instruction starts it, and it is stopped by the CLR instruction
      - These instructions start and stop the timers as long as GATE=0 in the TMOD register

GATE (cont.)

- The hardware way of starting and stopping the timer by an external source is achieved by making GATE=1 in the TMOD register

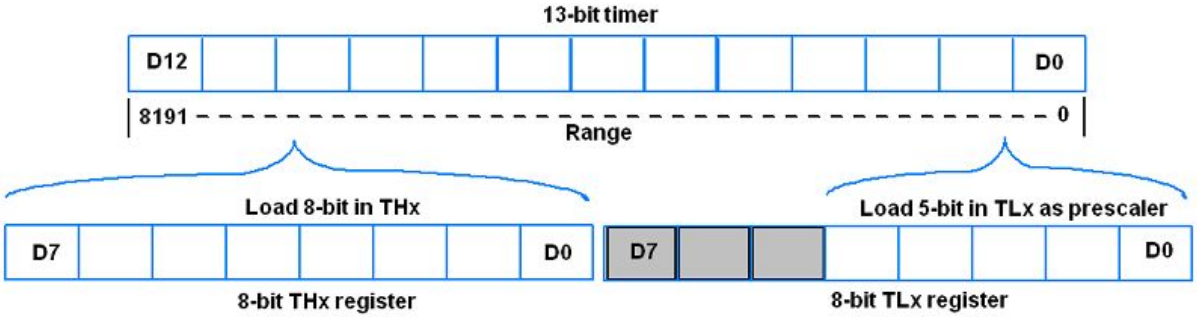
Example 9-3

Find the value for TMOD if we want to program Timer 0 in mode 2, use 8051 XTAL for the clock source, and use instructions to start and stop the timer.

Solution:

TMOD= 0000 0010 Timer 0, mode 2,
C/T = 0 to use XTAL clock source, and
gate = 0 to use internal (software)
start and stop method.

Mode 0 (13-bit timer mode): Mode 0 is a 13-bit timer mode for which 8-bit of THx and 5-bit of TLx (as Prescaler) are used. It is mostly used for interfacing possible with old MCS-48 family microcontrollers.



As shown in the above figure, 8-bit of THx and lower 5-bit of TLx used to form a total 13-bit timer. Higher 3-bits of TLx should be written as zero while using timer mode0, or it will affect the result.

Example: Let's generate a square wave of 2mSec period using an AT89C51 microcontroller with timer0 in mode0 on the P1.0 pin of port1. Assume xtal oscillator frequency of 11.0592 MHz. As the Xtal oscillator frequency is 11.0592 MHz we have a machine cycle of 1.085uSec. Hence, the required count to generate a delay of 1mSec. is,

$$\text{Count} = (1\times10^{-3})/(1.085\times10^{-6}) \approx 921$$

The maximum count of Mode0 is 2^13 (0 - 8191) and the Timer0 count will increment from 0 – 8191. So we need to load value which is 921 less from its maximum count i.e. 8191. Also, here in the below program, we need an additional 13 MC (machine cycles) from call to return of delay function. Hence value needed to be loaded is,

$$\text{Value} = (8191-\text{Count})+\text{Function_MCycles}+1 = 7284 = 0x1C74$$

So we need to load 0x1C74 value in Timer0.

1C74 = 000**1 1100 0111 0100** b, now load lower 5-bit in TL0 and next 8-bit in TH0

so here we get, TL0 = 000**1 0100** = 0x14 and TH0 = 1110 0011 = 0xE3

Mode 1 Programming

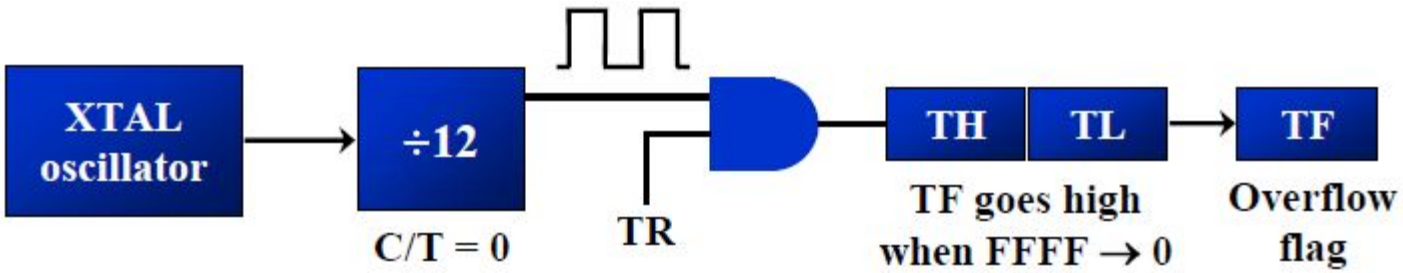
- The following are the characteristics and operations of mode1:
  - It is a 16-bit timer
    - It allows value of 0000 to FFFFH to be loaded into the timer's register TL and TH
  - After TH and TL are loaded with a 16-bit initial value, the timer must be started
    - This is done by SETB TR0 for timer 0 and SETB TR1 for timer 1
  - After being started, it starts to count up
    - It counts up until it reaches its limit of FFFFH

Mode 1 Programming (cont.)

- When it rolls over from FFFFH to 0000, it sets high a flag bit called TF (timer flag)
  - Each timer has its own timer flag: TF0 for timer 0, and TF1 for timer 1
  - This timer flag can be monitored
- When this timer flag is raised, one option would be to stop the timer with the instructions CLR TR0 or CLR TR1, for timer 0 and timer 1, respectively

◦ In order to repeat the process

- TH and TL must be reloaded with the original value
- TF must be reloaded to 0



Steps to Mode 1 Program

- Load the TMOD value register
  - Indicating which timer (timer 0 or timer 1) is to be used and which timer mode (1 or 2) is selected
- Load registers TL and TH with initial count value
- Start the timer
- Keep monitoring the timer flag (TF)
  - With the JNB TFx,target instruction to see if it is raised

Steps to Mode 1 Program (cont.)

- Get out of the loop when TF becomes high
- Stop the timer
- Clear the TF flag for the next round
- Go back to Step 2 to load TH and TL again

(a) in hex	(b) in decimal
(FFFF - YYXX + 1) × 1.085 µs where YYXX are TH, TL initial values respectively. Notice that values YYXX are in hex.	Convert YYXX values of the TH,TL register to decimal to get a NNNNN decimal number, then(65536 - NNNNN) × 1.085 µs

Figure 9-4. Timer Delay Calculation for XTAL = 11.0592 MHz

Example 9-4

In the following program, we are creating a square wave of 50% duty cycle (with equal portions high and low) on the P1.5 bit. Timer 0 is used to generate the time delay. Analyze the program.

```
            MOV    TMOD,#01          ;Timer 0, mode 1(16-bit mode)
HERE:       MOV    TL0,#0F2H         ;TL0 = F2H, the Low byte
            MOV    TH0,#0FFH         ;TH0 = FFH, the High byte
            CPL    P1.5              ;toggle P1.5
            ACALL  DELAY
            SJMP   HERE              ;load TH, TL again
;----------delay using Timer 0
DELAY:
            SETB   TR0               ;start Timer 0
AGAIN:      JNB    TF0,AGAIN         ;monitor Timer 0 flag until
                                     ;it rolls over
            CLR    TR0               ;stop Timer 0
            CLR    TF0               ;clear Timer 0 flag
            RET
```

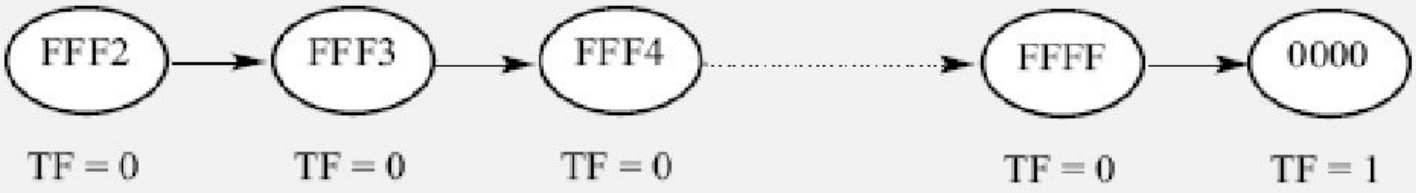
https://www.electronicwings.com/8051/8051-timers

Solution:

In the above program notice the following steps.

1. TMOD is loaded.
2. FFF2H is loaded into TH0 - TL0.
3. P1.5 is toggled for the high and low portions of the pulse.
4. The DELAY subroutine using the timer is called.
5. In the DELAY subroutine, Timer 0 is started by the "`SETB TR0`" instruction.
6. Timer 0 counts up with the passing of each clock, which is provided by the crystal oscillator. As the timer counts up, it goes through the states of FFF3, FFF4, FFF5, FFF6, FFF7, FFF8, FFF9, FFFA, FFFB, and so on until it reaches FFFFH. One more clock rolls it to 0, raising the timer flag (TF0 = 1). At that point, the JNB instruction falls through.
7. Timer 0 is stopped by the instruction "`CLR TR0`". The DELAY subroutine ends, and the process is repeated.

Notice that to repeat the process, we must reload the TL and TH registers and start the timer again.



Example 9-6

In Example 9-5, calculate the frequency of the square wave generated on pin P1.5.

Solution:

In the timer delay calculation of Example 9-5, we did not include the overhead due to instruction in the loop. To get a more accurate timing, we need to add clock cycles due to this instructions in the loop. To do that, we use the machine cycle from Table A-1 in Appendix A, as shown below.

```
                                        Cycles
HERE:  MOV  TL0,#0F2H                     2
       MOV  TH0,#0FFH                     2
       CPL  P1.5                          1
       ACALL DELAY                        2
       SJMP  HERE                         2
DELAY:
       SETB TR0                           1
AGAIN: JNB  TF0,AGAIN                    14
       CLR  TR0                           1
       CLR  TF0                           1
       RET                                2
                                 Total    28
```

T = 2 × 28 × 1.085 us = 60.76 us and F = 16458.2 Hz

Example 9-9

The following program generates a square wave on pin P1.5 continuously using Timer 1 for a time delay. Find the frequency of the square wave if XTAL = 11.0592 MHz. In your calculation do not include the overhead due to instructions in the loop.

```
            MOV   TMOD,#10H        ;Timer 1, mode 1(16-bit)
AGAIN:      MOV   TL1,#34H         ;TL1 = 34H, Low byte
            MOV   TH1,#76H         ;TH1 = 76H, High byte
                                   ;(7634H = timer value)
            SETB  TR1              ;start Timer 1
BACK:       JNB   TF1,BACK         ;stay until timer rolls over
            CLR   TR1              ;stop Timer 1
            CPL   P1.5             ;comp. P1.5 to get hi, lo
            CLR   TF1              ;clear Timer 1 flag
            SJMP  AGAIN            ;reload timer since Mode 1
                                   ;is not auto-reload
```

Solution:

In the above program notice the target of SJMP. In mode 1, the program must reload the TH, TL register every time if we want to have a continuous wave. Now the calculation. Since FFFFH – 7634H = 89CBH + 1 = 89CCH and 89CCH = 35276 clock count. 35276 × 1.085 μs = 38.274 ms for half of the square wave. The entire square wave length is 38.274 × 2 = 76.548 ms and has a frequency = 13.064 Hz.

Also notice that the high and low portions of the square wave pulse are equal. In the above calculation, the overhead due to all the instructions in the loop is not included.

Finding the Loaded Timer Values

- To calculate the values to be loaded into the TL and TH registers:
  - Assume XTAL = 11.0592 MHz
    - Divide the desired time delay by 1.085 us
    - Perform 65536 – n, where n is the decimal value we got in Step1
    - Convert the result of Step2 to hex, where yyxx is the initial hex value to be loaded into the timer's register
    - Set TL = xx and TH = yy

Example 9-10

Assume that XTAL = 11.0592 MHz. What value do we need to load into the timer's registers if we want to have a time delay of 5 ms (milliseconds)? Show the program for Timer 0 to create a pulse width of 5 ms on P2.3.

Solution:

Since XTAL = 11.0592 MHz, the counter counts up every 1.085 μs. This means that out of many 1.085 μs intervals we must make a 5 ms pulse. To get that, we divide one by the other. We need 5 ms / 1.085 μs = 4608 clocks. To achieve that we need to load into TL and TH the value 65536 - 4608 = 60928 = EE00H. Therefore, we have TH = EE and TL = 00.

```
          CLR   P2.3              ;clear P2.3
          MOV   TMOD,#01          ;Timer 0, mode 1 (16-bit mode)
HERE:     MOV   TL0,#0            ;TL0 = 0, Low byte
          MOV   TH0,#0EEH         ;TH0 = EE( hex), High byte
          SETB  P2.3              ;SET P2.3 high
          SETB  TR0               ;start Timer 0
AGAIN:    JNB   TF0,AGAIN         ;monitor Timer 0 flag
                                  ;until it rolls over
          CLR   P2.3              ;clear P2.3
          CLR   TR0               ;stop Timer 0
          CLR   TF0               ;clear Timer 0 flag
```

SJMP HERE

Mode 2 Programming

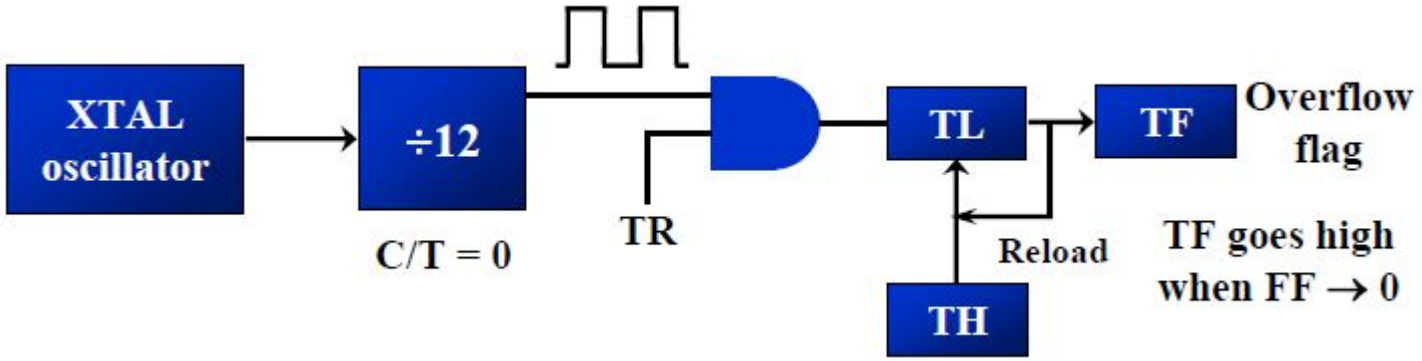
- The following are the characteristics and operations of mode 2:
  - It is an 8-bit timer
    - It allows only values of 00 to FFH to be loaded into the timer's register TH
  - After TH is loaded with the 8-bit value, the 8051 gives a copy of it to TL
    - Then the timer must be started
      - This is done by the instruction SETB TR0 for timer 0 and SETB TR1 for timer 1

Mode 2 Programming (cont.)

- After the timer is started, it starts to count up by incrementing the TL register
  - It counts up until it reaches its limit of FFH
  - When it rolls over from FFH to 00, it sets high the TF (timer flag)
- When TF is set to 1, TL is reloaded automatically with the original value kept by the TH register
- To repeat the process, we must simply clear TF and let it go without any need by the programmer to reload the original value

Mode 2 Programming (cont.)

- Mode 2 can auto-reload, in contrast with mode 1 in which the programmer has to reload TH and TL



Steps to Mode 2 Program

- Load the TMOD value register
  - Indicating which timer (timer 0 or timer 1) is to be used, and the timer mode (mode 2) is selected
- Load the TH registers with the initial count value
- Start timer
- Keep monitoring the timer flag (TF)
  - With the JNB TFx,target instruction to see whether it is raised

Steps to Mode 2 Program (cont.)

- Get out of the loop when TF goes high
- Clear the TF flag
- Go back to Step 4
  - Since mode 2 is auto-reload

Example 9-14

Assuming that XTAL = 11.0592 MHz, find (a) the frequency of the square wave generated on pin P1.0 in the following program, and (b) the smallest frequency achievable in this program, and the TH value to do that.

```
        MOV   TMOD,#20H        ;T1/mode 2/8-bit/auto-reload
        MOV   TH1,#5           ;TH1 = 5
        SETB  TR1              ;start Timer 1
BACK:   JNB   TF1,BACK         ;stay until timer rolls over
        CPL   P1.0             ;comp. P1.0 to get hi, lo
        CLR   TF1              ;clear Timer 1 flag
        SJMP  BACK             ;mode 2 is auto-reload
```

Solution:

(a) First notice the target address of SJMP. In mode 2 we do not need to reload TH since it is auto-reload. Now $(256 - 05) \times 1.085\ \mu s = 251 \times 1.085\ \mu s = 272.33\ \mu s$ is the high portion of the pulse. Since it is a 50% duty cycle square wave, the period T is twice that; as a result $T = 2 \times 272.33\ \mu s = 544.67\ \mu s$ and the frequency = 1.83597 kHz.

(b) To get the smallest frequency, we need the largest T and that is achieved when TH = 00. In that case, we have $T = 2 \times 256 \times 1.085\ \mu s = 555.52\ \mu s$ and the frequency = 1.8 kHz.

Example 9-16

Assuming that we are programming the timers for mode 2, find the value (in hex) loaded into TH for each of the following cases.

```
(a) MOV   TH1,#-200      (b) MOV   TH0,#-60
(c) MOV   TH1,#-3        (d) MOV   TH1,#-12
(e) MOV   TH0,#-48
```

Solution:

You can use the Windows scientific calculator to verify the result provided by the assembler. In Windows calculator, select decimal and enter 200. Then select hex, then +/- to get the TH value. Remember that we only use the right two digits and ignore the rest since our data is an 8-bit data.

Decimal	2's complement (TH value)
-3	FDH (253)
-12	F4H
-48	D0H
-60	C4H
-200	38H

The number 200 is the timer count till the TF is set to 1

The advantage of using negative values is that you don't need to calculate the value loaded to THx

Counter Programming

- Timers can also be used as counters
  - Counting events happening outside the 8051
  - A pulse outside of the 8051 increments the TH, TL registers
  - TMOD and TH, TL registers are the same as for the timer
    - Programming the timer also applies to programming it as a counter
      - Except the source of the frequency
  - The C/T bit in the TMOD registers decides the source of the clock for the timer

Counter Programming (cont.)

- When C/T = 1, the timer is used as a counter and gets its pulses from outside the 8051
- The counter counts up as pulses are fed from pins 14 and 15
  - These pins are called T0 (timer 0 input) and T1 (timer 1 input)

Table 9-1: Port 3 Pins Used For Timers 0 and 1

Pin	Port Pin	Function	Description
14	P3.4	T0	Timer/Counter 0 external input
15	P3.5	T1	Timer/Counter 1 external input

(MSB) (LSB)

Example 9-18

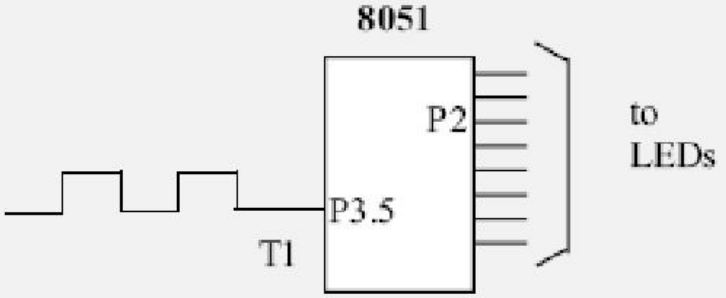
Assuming that clock pulses are fed into pin T1, write a program for counter 1 in mode 2 to count the pulses and display the state of the TL1 count on P2.

Solution:

```
            MOV   TMOD,#01100000B      ;counter 1, mode 2,C/T=1
                                       ;external pulses
            MOV   TH1,#0               ;clear TH1
            SETB  P3.5                 ;make T1 input
AGAIN:      SETB  TR1                  ;start the counter
BACK:       MOV   A,TL1                ;get copy of count TL1
            MOV   P2,A                 ;display it on port 2
            JNB   TF1,BACK             ;keep doing it if TF=0
            CLR   TR1                  ;stop the counter 1
            CLR   TF1                  ;make TF=0
            SJMP  AGAIN                ;keep doing it
```

Notice in the above program the role of the instruction "SETB P3.5". Since ports are set up for output when the 8051 is powered up, we make P3.5 an input port by making it high. In other words, we must configure (set high) the T1 pin (pin P3.5) to allow pulses to be fed into it.

P2 is connected to 8 LEDs and input T1 to pulse.



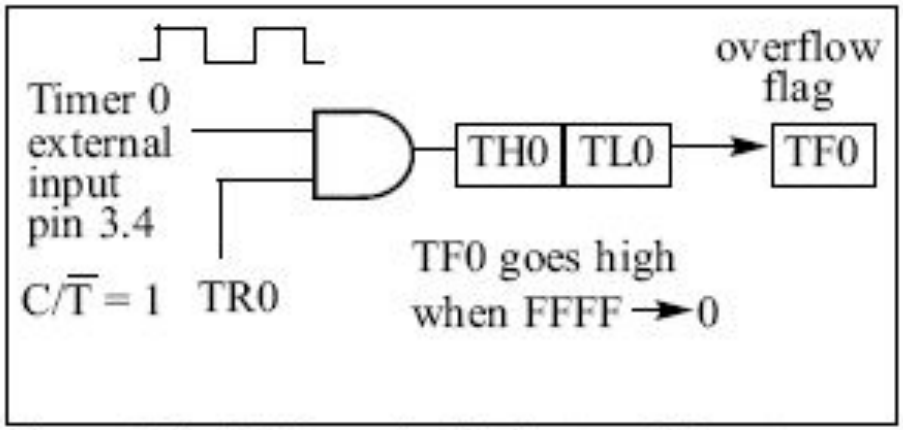
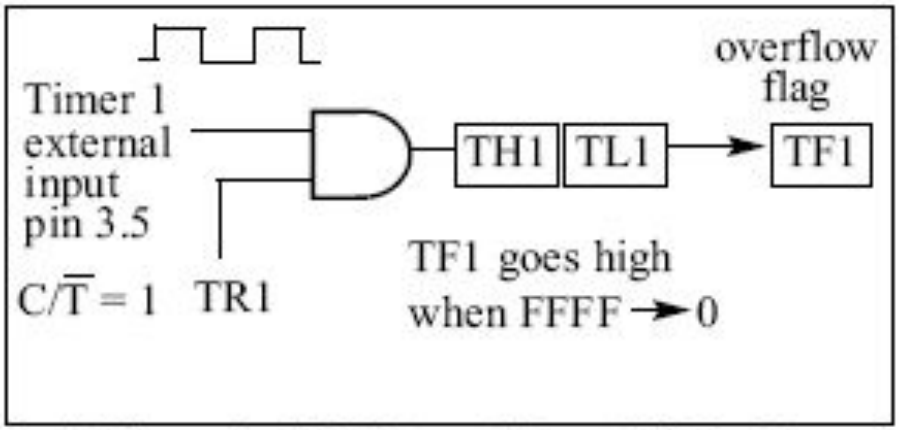


Figure 9-5. (a) Timer 0 with External Input (Mode 1)



(b) Timer 1 with External Input (Mode 1)

Case of GATE = 1

- The start and stop of the timer are done externally through pins P3.2 and P3.3 for timers 0 and 1, respectively
  - Allows to start or stop the timer externally at any time via a simple switch

